

SwiftPay Checkout: quick in-person payments for local businesses

Context

Local businesses on SwiftPay have no fast way to take a walk-in payment. Today the only public pay page is the invoice page (`apps/web/app/invoice/[publicId]/page.tsx`), which needs an invoice to be drafted first. Everything else (`/pay`, `/send`, the receive QR) is behind sign-in, and the receive QR on the dashboard even points customers at the *requester* screen (`/pay`) instead of a payer screen. A payer who signs in mid-flow also loses the amount, because `proxy.ts` drops the query string on redirect and no sign-in path reads `next` at all.

The feature: a merchant taps an amount, shows a QR (or shares a link), and the customer pays in seconds with **whatever they have**: a SwiftPay account, any crypto wallet on Arc, a card or bank account (Circle Onramp), or USDC on another chain (bridge). The merchant screen flips to "Paid" live. Also a permanent storefront QR (`/p/<username>`) for the counter or window.

Decisions already made with the user:

- Card/bank via Circle Onramp is in scope.
- Guest payments carry **no platform fee** (plain ERC-20 transfer, like invoices). SwiftPay-account payers keep the usual 0.1% via `/send`.
- Extras in scope: optional tip, print-ready QR poster, customer receipt, pay from other chains.

SwiftPay is **live on Arc mainnet**, so this ships into a production app that moves real money. The plan keeps every change additive (new tables, new routes, new pages), gates the few touches to live surfaces behind an env flag, and verifies on a testnet preview deploy before a mainnet smoke test with small real amounts.

Working names: routes `/c/<code>` (one charge), `/p/<username>` (storefront), `/business/checkout` (merchant). Server code under `apps/web/lib/checkout/`.

All paths below are relative to `apps/web/` unless they start with `packages/`.

Design summary

- **Everything is a charge.** The storefront page creates a `STOREFRONT` charge from the amount the customer types, then routes to `/c/<code>`. One payer page, one verifier, one matcher, one receipt.
- **Chain is the source of truth.** Wallet and SwiftPay paths post a tx hash; the server reads the receipt and credits only what actually arrived at the merchant wallet (same rule as `confirmInvoicePayment`). Onramp and bridge settlements arrive without a usable Arc tx hash, so the server scans incoming transfers to the merchant wallet from the charge's creation block and claims a matching transfer. A `tx_hash` `unique` constraint on the payments table makes claims race-safe.
- **Tip = excess.** A charge is PAID when received $\geq$ base amount. Anything above base is recorded as tip (the tip the customer selected pre-pay is only a hint for the total shown and for matching). Underpayments accumulate and the charge stays OPEN with "received X of Y".
- **State machine:** `OPEN` $\rightarrow$ `PAID` | `CANCELLED` | `EXPIRED`. Cancel only with no payments. Expiry is a sweep. A late receipt-based `/pay` on an EXPIRED charge is still accepted (money that arrived must never be unattributed). All updates use guarded `.eq("status", ...)` like `lib/account/service.ts:1194-1232`.
- **Charge code:** 10 chars Crockford base32 from 8 random bytes, uppercase; lookup normalises `O` $\rightarrow$ `0`, `I/L` $\rightarrow$ `1`. Retry on 23505.

Phase 1: data, public charge page, wallet + SwiftPay paths, merchant screen

1a. SQL migration

New packages/database/supabase/business-checkout.sql, following account-business.sql (uuid pk, wallet_address references profiles, named checks, RLS on, <table>_no_client_access policy, grants only to service_role):

- business_charges: id, public_id unique, wallet_address, kind (MERCHANT|STOREFRONT), status (OPEN|PAID|EXPIRED|CANCELLED), currency (USDC|EURC), amount, tip_amount default '0', amount_received default '0', overpayment default '0', note, payer_wallet, pending_method (WALLET|SWIFTPAY|ONRAMP|BRIDGE|null), pending_ref, pending_started_at, reported_amount, created_block bigint, expires_at, paid_at, created_at, updated_at. Indexes: (wallet_address, created_at desc); partial (expires_at) where status='OPEN'; partial (wallet_address, created_block) where status='OPEN' and pending_method in ('ONRAMP','BRIDGE').
- business_charge_payments: id, charge_id, tx_hash unique, amount, asset, source (SWIFTPAY|WALLET|ONRAMP|BRIDGE|RECONCILE), matched_by (RECEIPT|SCAN), payer_wallet, block_number, status default 'CONFIRMED', paid_at, created_at.
- business_checkout_scan_cursors: (chain_id, wallet_address) pk, last_block, updated_at (shape of incoming-transfer-cursors.sql).
- Re-create account_activity_source_check adding 'checkout' (and 'points', which the TS list in lib/activity/types.ts already has but the DB constraint lacks).

Expiry: MERCHANT 24h, STOREFRONT 6h; registering an ONRAMP/BRIDGE intent bumps to 72h.

Modify lib/account/db.ts: add charges, chargePayments, checkoutCursors to accountTables; make readAccountDbError mention business-checkout.sql when the message contains business_charge.

1b. Server library lib/checkout/

File	Purpose / what to reuse
types.ts	ChargeRecord, ChargePaymentRecord, enums, PublicCharge (payer-safe: no payer_wallet, no pending_ref).
codes.ts (pure)	generateChargeCode(), normalizeChargeCode().
money-rules.ts (pure)	limits (min 0.5, max 50 000 merchant / 10 000 storefront, tip ≤ min(3×amount, 5 000)), validateChargeAmount, validateTip, applyPaymentToCharge({charge, received}) → {status, amountReceived, tipAmount, overpayment}, isExpired. Use moneyNumber/roundMoney from lib/account/money.ts.
verify.ts	receivedChargeAmount({logs, currency, destination}) (pure): USDC = receivedAmount from lib/account/verify-invoice-payment.ts on the ERC-20 address; only if that is null , sum logs from NATIVE_USDC_EVENT_ADDRESS at 18 decimals (Arc emits both for ERC-20 sends; never count both). verifyChargeTransfer({txHash, currency, destination}) fetches the receipt via createArcRpcClient() and returns {amount, from, blockNumber}
settlement.ts	claimTransferForCharge({charge, txHash, amount, from, blockNumber, source, matchedBy}): insert payment row (23505 → already claimed), reject hash present in business_invoice_payments, recompute via applyPaymentToCharge, guarded update, then createSavingsNotificationResult({kind:"payment_received", relatedTxHash, ownerWallet, title:"Charge <code> paid", metadata:{type:"charge_paid", chargeId, code, amount, tip, source}}) from lib/save/notifications.ts:286.
service.ts	Merchant: createCharge (reads created_block from getBlockNumber()), listCharges, getChargeForOwner, cancelCharge, chargeSummary(wallet). Public: getPublicCharge(code) (business card via loadBusinessProfile/loadAccount like getPublicInvoice at lib/account/service.ts:1034), setChargeTip, registerChargeIntent, confirmChargePayment({code, txHash, payerWallet?}) (mirror confirmInvoicePayment L1104), getPublicStorefront(username) (resolve via resolvePaymentIdentity in lib/business/service.ts:703), createStorefrontCharge.

File	Purpose / what to reuse
errors.ts	Spread accountErrors (lib/account/errors.ts) and add chargeNotFound, chargeNotOpen, chargeNotOwned, invalidCharge(msg), storefrontNotFound, onrampUnavailable.
client.ts	Browser fetchers in the style of lib/account/client.ts:232-274.

1c. API routes

All routes: `export const runtime = "nodejs"`, params is a Promise, errors via `jsonBusinessError`, success via `jsonOk` (lib/business/http.ts), public ones Cache-Control: no-store.

Merchant (auth `readActor(request, body) → requireBusinessAccount`, records under `businessWallet`):

- `POST /api/business/checkout/charges {ownerWallet, amount, currency?, note?} + readIdempotencyKey → {charge}; rate limit checkout-create:<wallet> 120/h.`
- `GET /api/business/checkout/charges?status=&page= → {charges, summary}.`
- `GET /api/business/checkout/charges/[id] → {charge}` (Phase 2: runs the throttled scan when pending ONRAMP/BRIDGE); `checkout-poll:<wallet> 60/min.`
- `POST /api/business/checkout/charges/[id]/cancel → {charge}.`

Public (no auth; rate limits keyed on first hop of `x-forwarded-for` + code, via `consumeRateLimit`):

- `GET /api/checkout/charges/[code] → {business, destinationWallet, charge}; 120/min per IP.`
- `POST /api/checkout/charges/[code]/tip {tip} (OPEN only); 30/min.`
- `POST /api/checkout/charges/[code]/intent {method, payerWallet?, reference?}; 30/min.`
- `POST /api/checkout/charges/[code]/pay {txHash, payerWallet?}; source SWIFTPAY when getSessionOwnerWallet() resolves, else WALLET; amount comes from the receipt only; 20/min.`
- `GET /api/checkout/storefront/[username] → {business, destinationWallet, currency}; 60/min.`
- `POST /api/checkout/storefront/[username]/charges {amount, tip?, currency?} → {code}; 10/min per IP, 60/h per merchant.`

Security rules: destination wallet always from the DB row, never the body; `PublicCharge` carries no PII; tx hash regex `^0x[a-f0-9]{64}$`; public POSTs already pass the proxy's same-origin check.

1d. Proxy next fix and sign-in destination

- `proxy.ts:167-171`: set next to `pathname + request.nextUrl.search` and `url.hash = "sign-in"` (the landing page opens the modal on `#sign-in`, `components/landing/landing-page.tsx:217`). Keep `/c` and `/p` out of `protectedRouteMatchers` and `config.matcher`.
- New `lib/sign-in-destination.ts`: `readSafeNextPath()` (relative path only, no `//`, scheme, `\`, `..`), `rememberNextPath()` / `consumeNextPath()` in `sessionStorage` (survives the Google OAuth round trip), `resolveSignInDestination({accountTypeSelected, isBusiness, next}) → next` when the account is already set up, else `/onboarding` | `/business` | `/dashboard`.
- Use it at the three places that compute destination: `components/email-sign-in.tsx:220-229`, `components/circle-google-login.tsx:735-755`, `components/landing/sign-in-panel.tsx:47,134-148`; call `rememberNextPath()` in the landing page mount effect.

1e. Dashboard settle hook (app/dashboard/page.tsx)

- Add "charge" to the `hasPaymentPrefill` key list (L669).
- Read `charge` next to `invoicePublicId` (L673); load `linkedCharge` via `fetchPublicCharge` (mirror the invoice effect ~L1455).
- Add `settleChargePayment(txHash)` beside `settleInvoicePayment` (L1478): retry loop calling `payPublicCharge(code, {txHash, payerWallet})`, de-duped by a ref.

- Call it where `settleInvoicePayment` is called (~L2352), which already covers Circle and external sends.
- Payer link built by `/c/<code>`:
`/send?to=<wallet>&amount=<total>&token=<cur>&memo=<code>&charge=<code>` (to/amount/token/memo are consumed ~L1356-1390).

1f. Pages and components

Public payer:

- `app/c/[code]/page.tsx` → `components/checkout/pay-charge-page.tsx`: business card, amount, `TipPicker`, payment-method chooser, poll `fetchPublicCharge` every 2.5s while OPEN, PAID state → `ChargeReceiptCard` + a plain "Get SwiftPay" link (no registration, no tracking). Fire `trackTractionEvent({eventType:"payment_submitted", source:"checkout", ...})`.
- `components/checkout/tip-picker.tsx`: presets 0/10/15/20% + custom, debounced `updateChargeTip`.
- `components/checkout/pay-with-wallet.tsx`: lift of the invoice page's pay flow (`app/invoice/[publicId]/page.tsx:88-320`): `WalletConnectButton`, `switchToArc`, `balanceOf`, `writeContractAsync` `transfer(getAddress(dest), parseUnits(total, 6))`, `sessionStorage` pending key `swiftpay:charge-payment:<code>`, `waitForTransactionReceipt`, `registerChargeIntent({method:"WALLET"})` before signing, then `payPublicCharge` with 4 retries. `recordPlatformTransactionActivity` (source "checkout") only when `fetchWalletSessionForAddress(address).authenticated`.
- `components/checkout/pay-with-swiftpay.tsx`: the `/send?...&charge=` link (works signed-out thanks to 1d).
- `app/p/[username]/page.tsx` → `components/checkout/storefront-page.tsx`: business card, shared `components/checkout/amount-keypad.tsx`, optional tip, Continue → `createStorefrontCharge` → `router.push("/c/<code>")`.

Merchant:

- `app/business/checkout/page.tsx`: `PlatformAccessGate` + `PlatformChrome` (pattern `app/business/invoices/page.tsx`), business-only via `useOptionalAccount()?.isBusiness`.
- `components/checkout/merchant-checkout-hub.tsx`: `AmountKeypad` + note + Charge → large `LazyQRCodeSVG` of `${origin}/c/<code>`, copy / `navigator.share`, poll `fetchCharge` every 2.5s; on PAID show `showSuccess()` (`components/success-popup.tsx`) plus an in-place green state with amount and tip; "New charge" resets. Below it `RecentChargesList` (status chip, amount, time, explorer link, cancel). Second tab "Storefront QR" (poster button lands in Phase 4).

Also: add "checkout" to `activitySources` in `lib/activity/types.ts`.

Phase 2: card/bank via Circle Onramp, and hash-less matching

- `lib/checkout/match.ts` (pure) `selectMatches({transfers, charges, headBlock, claimedHashes})`: same symbol; `blockNumber ≥ created_block`; `headBlock - blockNumber ≥ 120` grace so receipt-based settles win first; hash not claimed; sender not a payer_wallet of the merchant's open charges; amount within `max(0.05, 1%)` of amount + tip_amount, or of `reported_amount` when set; transfers oldest-first, pick the closest-amount charge, tie → oldest intent; one-to-one.
- `lib/checkout/scan.ts` `syncChargeMatches(wallet)`: throttle 15s via `cursor.updated_at` (pattern `lib/notifications/incoming-payments.ts:38-90`); skip when no matchable charges; range from cursor or `min(created_block)`, capped at 43 200 blocks per run; `fetchIncomingTransfersFromRpc` (`lib/arc-transfers.ts:127`); claim via `claimTransferForCharge`; write the cursor only after all claims succeed.
- `lib/checkout/sweep.ts`: `expireStaleCharges()`, `catchUpChargeScans()`.
- Routes: POST `/api/checkout/charges/[code]/settled` {amount?, tokenSymbol?, reference?} stores `reported_amount` then runs the scan (10/min); POST `/api/checkout/charges/[code]/onramp-session` {} → Circle session with `destinationAddress = charge.wallet_address`, `appUserId = "charge:<code>"`, chain per `isArcMainnet()` (copy `app/api/onramp/sessions/route.ts:34-96`), sets intent ONRAMP, 503 when

ONRAMP_API_KEY unset; 5/h per code, 20/h per IP. POST /api/business/checkout/charges/[id]/reconcile {txHash} for the merchant fallback (any verified transfer to their wallet, source RECONCILE).

- Wire syncChargeMatches into the merchant GET .../[id] and public GET .../[code] when the charge is OPEN with an ONRAMP/BRIDGE intent, so polling drives matching. Scans hit ARC_SERVER_RPC_URL (the dedicated mainnet endpoint), are throttled to one per merchant per 15s, and skip merchants with no matchable charges, so polling adds no RPC load for wallet/SwiftPay payments.
- components/checkout/pay-with-onramp.tsx: adapt components/deposit/onramp-panel.tsx (createOnrampKit, parseOnrampSession, kit.openWindow synchronously on tap, mountIframe fallback); onDepositSettled({payload:{amount, tokenSymbol}}) → reportChargeSettled; copy that bank transfers can take a while and the page can be closed. Hide the option when GET /api/onramp/sessions reports enabled:false. On mainnet the session carries assets: { pairs: [{ token: "USDC", chain: "arc" }] } exactly as the existing route does (app/api/onramp/sessions/route.ts:80-85), so EURC charges do not offer card/bank.
- Cron app/api/cron/checkout/route.ts (isCronAuthorized, maxDuration = 60) running both sweeps; add { "path": "/api/cron/checkout", "schedule": "0 10 * * *" } to vercel.json.

Phase 3: pay from USDC on other chains

- components/checkout/pay-with-bridge.tsx: adapt components/deposit/DepositPanel.tsx:170-260. Source chain from the DepositSourceChain list in lib/deposit-to-arc.ts (mainnet: Base, Ethereum, Arbitrum, Optimism, Avalanche, Polygon; it already switches by isArcMainnet()), registerChargeIntent({method:"BRIDGE"}), then bridgeUsdcToArc({amount: total, provider, recipientAddress: destinationWallet, source, onStep}) (lib/deposit-to-arc.ts:355). If the result is not pending and has an Arc-side hash, call payPublicCharge; otherwise reportChargeSettled({reference: sourceTxHash}) and rely on the scan. Switch the wallet back to Arc in finally.

Phase 4: extras and entry points

- Poster: lift svgToPngBlob from components/dashboard/receive-share-card.tsx:18-47 into lib/qrcode-image.ts; lib/checkout/poster.ts draws a 1240×1754 canvas with drawSwiftPayBrand / loadBrandImage (lib/brand-canvas.ts), business name/logo, "Scan to pay with SwiftPay", the QR, and the URL; download PNG from the Storefront QR tab.
- Receipt: lib/checkout/receipt.ts downloadChargeReceipt as a sibling of lib/account/invoice-receipt.ts (move roundRect into lib/brand-canvas.ts); explorer link via explorerTxUrl (lib/onchain-facts.ts).
- Entry points: components/business/overview/business-quick-actions.tsx add "Charge a customer" → /business/checkout; components/dashboard/quick-actions.tsx add {href:"/business/checkout", icon: Store, label, businessOnly:true} + label; app/manifest.ts shortcut "Get paid" → /business/checkout.
- i18n: navLabelKeys["/business/checkout"] and a /business/checkout branch in pageCopyForPath **before** the generic /business/ fallback (lib/i18n/index.ts:85-141); add nav.checkout, pages.checkoutTitle/Subtitle to en.ts and every other locale file (they are typed Messages).
- Receive QR fix: components/dashboard/receive-share-card.tsx:71-77 → \${origin}/p/\${username} when a username exists.
- Overview stats: getBusinessOverview (lib/account/service.ts:668) adds checkout: chargeSummary(wallet) (todayCount, todayVolume, todayTips, openCount); one card in components/account/business-overview.tsx.
- .env.example (root and apps/web): comment that Checkout card/bank uses ONRAMP_API_KEY / ONRAMP_REFERRER_DOMAIN, and scans use ARC_SERVER_RPC_URL. One new key, NEXT_PUBLIC_CHECKOUT_ENABLED (see Rollout).

Rollout on the live app

Everything new is additive, so the order below never breaks a running deployment:

- 1 **Migration first.** `business-checkout.sql` only creates tables and re-creates the `account_activity_source_check` constraint (a superset of today's values). It is idempotent and safe to run in the Supabase SQL editor before the code deploys. `readAccountDbError` names the file if the tables are missing.
- 2 **Feature flag for live surfaces.** `NEXT_PUBLIC_CHECKOUT_ENABLED` (default off) gates the touches to pages people use today: the business and dashboard quick actions, the manifest shortcut, the receive-QR target, and the overview card. The new routes and pages are always deployed but only reachable by URL until the flag is on. The proxy `next` fix and the sign-in destination helper ship unflagged: they are bug fixes and default to today's behaviour when `next` is absent.
- 3 **Testnet preview.** A preview deploy with `NEXT_PUBLIC_ARC_NETWORK=testnet` (the app already supports the flip, README "Going live on Arc mainnet") runs the full manual checklist below with test USDC, including the Circle Onramp sandbox and a Base Sepolia bridge.
- 4 **Mainnet smoke test.** With the flag still off in production, a real business account creates a charge of about 0.50 USDC and pays it once by wallet and once by SwiftPay account. Confirm the payment row, the notification, and the activity entry. Then enable the flag.
- 5 **Watch.** For the first days, check `business_charges` for OPEN rows older than their expiry (sweep health) and `business_charge_payments` with `matched_by = 'SCAN'` (matcher health). The merchant reconcile action is the manual fallback if a real payment is not matched.

Verification

Unit tests (node:test, .test.mjs, TS imports via `--import ./lib/payment-engine/__tests__/register.mjs`):

- `lib/checkout/__tests__/codes.test.mjs`: alphabet, length, O/I/L normalisation.
- `lib/checkout/__tests__/money-rules.test.mjs`: bounds, tip cap, `applyPaymentToCharge` (exact, excess-as-tip, underpayment stays OPEN, cumulative).
- `lib/checkout/__tests__/verify.test.mjs`: ERC-20 log credited; native `0xff...fe` credited only when no ERC-20 log; both present counted once; wrong destination → null (reuse the log builder in `lib/account/__tests__/verify-invoice-payment.test.mjs`).
- `lib/checkout/__tests__/match.test.mjs`: tolerance, created-block bound, grace window, claimed hash skipped, payer-wallet exclusion, closest amount then FIFO, one-to-one, `reported_amount` hint.
- `lib/__tests__/sign-in-destination.test.mjs`: rejects `//evil`, `https://...`, `/...`; accepts `/send?...&charge=...`
- `apps/web/package.json`: add `test:checkout` and chain it into `test`.

Commands:

```
cd apps/web && pnpm run typecheck && pnpm run test:checkout && pnpm run test:account
```

Manual end-to-end, first on the testnet preview deploy (steps 1 to 7), then steps 1 to 3 and 6 on mainnet with amounts around 0.50 USDC (apply `business-checkout.sql` first):

- 1 Business account → `/business/checkout` → charge 5.00 → QR shown, recent list shows OPEN.
- 2 Private window → `/c/<code>` → tip 1.00 → any wallet (MetaMask on Arc) → transfer 6.00 → payer page flips PAID and downloads a receipt; merchant screen turns green with "+1.00 tip" within ~3s; bell shows "Charge ... paid".
- 3 Signed-out SwiftPay user → "Pay with SwiftPay" → lands on `/?next=/send?...&charge=...#sign-in`, modal opens → sign in → `/send` prefilled → send → charge PAID. Repeat with a Google (Circle) wallet.
- 4 Card/bank → Circle Onramp widget (sandbox on the preview) → complete → charge flips PAID once the transfer is ~120 blocks old and a poll runs; close the tab, reopen the merchant page, confirm it still matched. On mainnet, one real card purchase for the smallest amount the widget allows.

- 5 Bridge: wallet with USDC on Base (Base Sepolia on the preview) → "Pay from another chain" → PAID via receipt or scan.
- 6 Storefront: `/p/<username>` → 3.50 → `/c/<code>` → pay by wallet. Download the poster and scan it with a phone.
- 7 Negatives: 3.00 on a 5.00 charge stays OPEN ("received 3.00 of 5.00"); replaying a hash on another charge is rejected; an expired charge refuses new intents but a late `/pay` still marks it PAID; merchant reconcile with a valid hash to their wallet works.
- 8 Regression on live paths: with the flag off, the dashboard receive QR, quick actions, and sign-in redirects behave exactly as before; with it on, the receive QR scans to `/p/<username>`.

Open risks

- **Onramp KYC friction and coverage.** Circle Onramp verifies identity per `appUserId`; with a pseudonymous `charge:<code>` every guest re-verifies, and coverage is US/EU. Show the option only when configured and label it honestly. Sandbox settlement on the preview may differ from production, so the mainnet smoke test includes one real small card purchase.
- **Real funds during testing.** Every mainnet test moves real USDC to the test business wallet; keep amounts small and reuse one business account so the money stays in hand.
- **Onramp amount is typed in the widget**, not preset by us, so mismatches beyond tolerance need the merchant reconcile fallback.
- **Bridge forwarder fees** on mainnet may reduce the minted amount; the tolerance covers small charges and the receipt path credits the exact mint when a hash is available.
- **Same-amount collisions** at busy counters (two open 5.00 onramp charges) resolve by closest amount then oldest intent. Wallet and SwiftPay paths are unaffected.
- **Notification dedupe** on `related_tx_hash`: whichever of the bell's "payment received" sync or the "charge paid" notification inserts first wins.